

Automated Approach for System-level Testing of Unmanned Aerial Systems

Hassan Sartaj

*Quest-UAV Dependability Lab, Department of Computer Science
National University of Computer and Emerging Sciences
Islamabad, Pakistan*

ORCID: <https://orcid.org/0000-0001-5212-9787>

Abstract—Unmanned aerial systems (UAS) have a large number of applications in civil and military domains. UAS rely on various avionics systems that are safety-critical and mission-critical. A major requirement of international safety standards is to perform rigorous system-level testing of avionics systems, including software systems. The current industrial practice is to manually create test scenarios, manually or automatically execute these scenarios using simulators, and manual evaluation of the outcomes. A fundamental part of system-level testing of such systems is the simulation of environmental context. The test scenarios typically consist of setting certain environment conditions and testing the system under test in these settings. The state-of-the-art approaches available for this purpose also require manual test scenario development and manual test evaluation. In this research work, we propose an approach to automate the system-level testing of the UAS. The proposed approach (AITester) utilizes model-based testing and artificial intelligence (AI) techniques to automatically generate, execute, and evaluate various test scenarios. The test scenarios are generated on the fly, i.e., during test execution based on the environmental context at runtime. We develop a toolset to support automation. We perform a pilot experiment using a widely-used open-source autopilot, ArduPilot. The preliminary results show that the AITester is effective and efficient in violating autopilot expected behavior.

Index Terms—Model-based Testing, Artificial Intelligence, Reinforcement Learning, Unmanned Aerial Systems, Autopilot

I. INTRODUCTION

Unmanned aerial systems (UAS) mainly consist of an unmanned aerial vehicle (UAV) and a ground control station (GCS). A UAV is an aircraft that does not have a human pilot on-board. It is either operated remotely from the GCS or autonomously using the autopilot system. UAS are becoming progressively important due to their large number of applications in a variety of domains. For example, UAS are used for rescue operations, survey and monitoring, agriculture and farming, weather forecast, airstrike, military combat, etc. UAS rely on several avionics software systems (e.g., an autopilot, navigation system, communication system, etc.) that are critical for completing the mission or are critical to safety. Malfunctioning of the UAS avionics software can lead to mission failure that can result in human life and financial loss. To ensure the dependability of such systems, international safety standards such as DO-178C [1] and STANAG 4586 [2] define several guidelines. The automated testing of avionics systems is essential due to the huge amount of cost and effort required to follow these guidelines.

UAS are a special type of cyber-physical system (CPS) and are also considered context-aware systems (CAS). The system constraints and environment parameters of UAS are different from other types of CPS/CAS, e.g., the autopilot control mechanism and the operating environment of autonomous cars are completely different than that of UAS. The system-level testing of UAS requires the simulation of the UAV flight behavior with the help of flight simulators. It is typically performed at three levels, (i) software-in-the-loop (SIL) testing, (ii) hardware-in-the-loop (HIL) testing, and (iii) pilot-in-the-loop (PIL) testing also known as field testing. In SIL testing, the behavior of avionics software components and the environment are simulated. In HIL testing, the original avionics hardware parts are used and the environment is simulated. In PIL testing, UAS are tested in a real environment. Testing at each level highly depends on the environmental context, i.e., test scenarios created for SIL testing may not be useful for HIL/PIL testing. The main challenge in automating system-level testing of the UAS is the generation and evaluation of environmental context-aware test scenarios.

A common practice is to manually create various test scenarios by setting certain environmental conditions, manually write simulator scripts for each test scenario, manually or automatically execute these scenarios, and analyze the outcomes. The existing approaches available in literature also require manual test scenario development and manual test evaluation (e.g., [3]–[6]). Only a limited set of potential scenarios can be tested manually. Furthermore, the activity of manual testing is tedious and error-prone due to a large number of potential scenarios that need to be tested in an extremely dynamic environment of the UAV.

In this research work, we propose an approach to automate the system-level testing of UAS, initially aimed at the SIL level. Our approach (AITester) utilizes model-based testing (MBT) and artificial intelligence (AI) techniques to automatically generate, execute, and evaluate various test scenarios. The test scenarios are generated on the fly, i.e., during test execution based on the environmental context at runtime. We provide a toolset to support testing automation. We perform a pilot experiment on a widely-used open-source autopilot, ArduCopter. The preliminary results show that the AITester is effective and efficient in generating and executing test scenarios and evaluating the expected behavior of autopilot.

II. REVIEW OF THE STATE-OF-THE-ART

This section presents the review of the state-of-the-art works covering various aspects of UAS such as UAV and autopilot testing, and the reinforcement learning-based testing of UAVs.

A. UAV and Autopilot Testing

Johnson and Fontaine [7] developed a hardware and software-in-the-loop platform to carry out flight testing of small UAVs. Sorton and Hammaker [8] manually created fifteen test scenarios for testing UAV in the hardware and software-in-the-loop simulation. How et al. [4] developed a real-time test environment that can incorporate multiple autonomous (aerial and ground) vehicles. The above-mentioned techniques require manual effort in the generation or execution or evaluation of test scenarios, however, our approach can automatically generate, execute, and evaluate flight test scenarios. Keith and Paul [9] developed a testing environment to test UAV missions in a real-world context. Our approach targets system-level testing of UAS rather than UAV missions.

Mullins et al. [10] proposed an approach to generate challenging test scenarios for testing autonomous vehicles while utilizing a number of manually designed test scenarios. Our approach does not require base test cases to generate new test cases. Schulte and Spencer [11] presented a model-based approach for UAVs that identify faults, sources of faults, and performs action accordingly during autonomous flight using manually designed flight tests. Our approach targets automated system-level testing but does not perform onboard diagnostics during the UAV flight. Ma et al. [12] proposed an uncertainty-based modeling framework to test the self-healing part of the CPS. The proposed approach was used to test the self-healing behavior of the UAV, whereas our approach automates the system-level testing of the UAS.

Ulbig et al. [13] proposed an approach and various unit and system-level test scenarios for the testing of the Terrain Awareness and Warning System of the UAV. Our approach automatically generates, executes, and evaluates test scenarios. Sarkar et al. [14] proposed an approach and tool to automatically perform the flight testing of the UAV using flight data and manual test oracle. Since the approach highly depends on the flight data, the inconsistency in data and labeling error may lead to unpredictable flight testing. In contrast, our approach does not rely on flight data. De et al. [15] manually designed various flight test scenarios for testing the vertical takeoff and landing behavior of the UAV carrying different payloads. Rizk et al. [16] developed a GNSS-based emulator to facilitate UAV testing without flying. The emulator is required to be configured manually for testing the UAV. Wu et al. [5] manually created various test scenarios for testing multi-rotor UAVs under different wind conditions and various flight modes (e.g., hover and flip). Yang et al. [17] manually developed test cases to test the acoustic behavior of the UAV. Scanavino et al. [6] created a test environment to evaluate the performance of the UAV under different climate conditions. To use the testing environment, the flight test scenarios are needed to be developed manually. In comparison to these works, our

approach can automatically perform the system-level testing of the UAS.

To test UAV autopilot, Ambroziak and Gosiewski [18] manually defined various flight tests to analyze the variation in airspeed, altitude, and trajectories followed by the UAV. Erdos and Watkins [19] manually created flight test scenarios to test the autopilot for the roll, pitch, altitude, airspeed, and navigation performance. Hartley et al. [20] manually developed seven flight test scenarios to analyze the waypoint navigation of the autopilot. In all the above-mentioned works, the test scenarios are created manually. Whereas our approach automatically performs system-level testing of the UAV. Gafford et al. [21] proposed an approach for resolving feature interaction conflicts in cyber-physical systems including UAVs, whereas our approach targets automated system-level testing of UAS.

B. Reinforcement Learning-based UAV Testing

Junell et al. [22] devised an approach using reinforcement learning (RL) to reduce the number of trials during flight testing of a quadrotor. Siddiquee et al. [23] designed a vision-based RL agent to test the guidance system of a quadcopter. Ma et al. [24] developed two algorithms based on RL to test the self-healing part of CPS under different uncertainties. The proposed algorithms were used to test the self-healing behavior of the ArduCopter. The main difference between these works with our work is that our approach automates system-level testing of the UAS.

III. MOTIVATION OF THE WORK

A traditional method of testing UAS at the system level includes a human tester (pilot) who knows UAV flight behavior and has expertise with UAV flight simulators. The tester is provided with the system under test (SUT) and the specifications of the SUT. Using this information, the tester manually designs test scenarios. For each test scenario, the tester simulates the behavior of a UAV in a particular environment with the help of a flight simulator, and at each flight operation, observes the behavior of the SUT by comparing the expected behavior mentioned in specifications. Testing in this way usually requires several days and only a limited set of potential scenarios can be tested. Furthermore, it is tedious and error-prone activity.

To automate the manual testing of UAS, a human-level control on a flight simulator and a mechanism to model the SUT specifications are required. For this purpose, we use RL to get human-level control on a flight simulator and model-based testing (MBT) to model the specifications of SUT. RL is an AI technique that aims at training an AI agent to explore the environment by performing various actions. The actions are generally selected considering the previous experiences and future outcomes. Since the UAV flight operations are inter-dependent, in our case, it is important to consider previously executed actions, the current state of the UAV, and the future impact of selecting a new action. The purpose of using MBT is that it provides a systematic method to create test-ready models covering various aspects of the SUT [25].

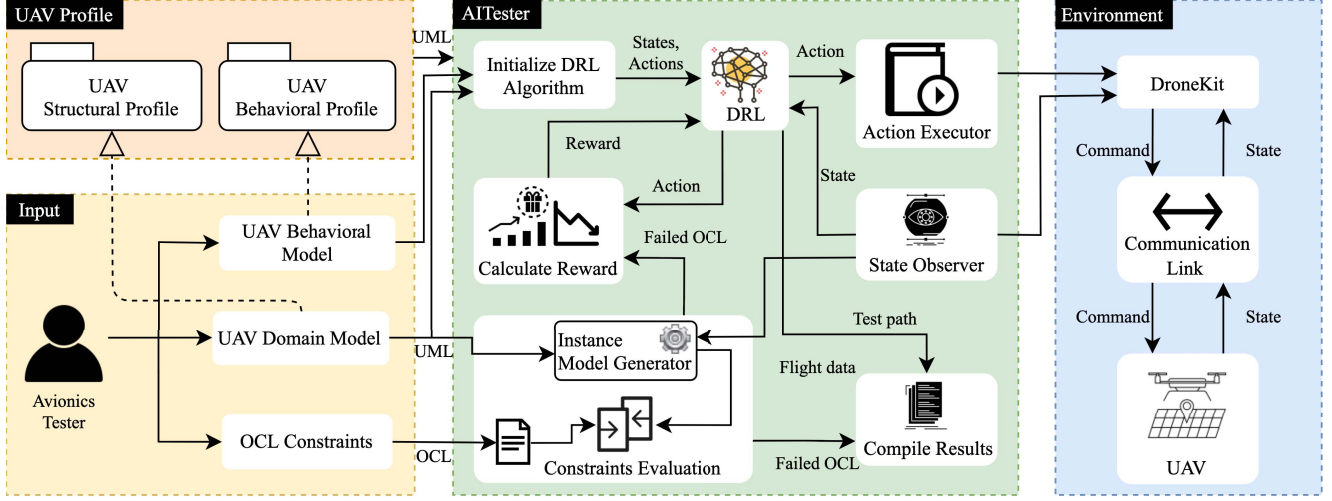


Fig. 1. Overall workflow of the proposed approach

IV. CONTRIBUTIONS

The primary contributions of this research work are summarized below.

- We present a comprehensive modeling methodology for UAVs. This includes profiles (developed using Unified Modeling Language (UML) [26]) for modeling the structural and behavioral aspects of a UAV and modeling guidelines for avionics testers.
- We propose an approach (AITester) that utilizes MBT and AI to automatically generate, execute, and evaluate various test scenarios based on the environmental context at runtime.
- We develop a toolset that implements our proposed approach to support testing automation. The toolset is available at the GitHub repository¹.
- We plan to perform an empirical evaluation of the proposed approach on a number of case studies representing different subsystems of UAS such as Autopilot and GCS.

V. OVERVIEW OF THE PROPOSED APPROACH

Figure 1 shows the overall workflow of the proposed approach. The first phase of our approach is to create test-ready models using the specifications of SUT. For this purpose, we propose two UML profiles, UAV structural profile, and UAV behavioral profile. UAV structural profile models domain concepts and properties of the UAV and its subsystems such as autopilot. UAV behavioral profile models the flight behavior of the UAV including abstract flight states and events. The avionics tester is required to model the domain of the UAV SUT using the structural profile and the flight behavior of the UAV using the behavioral profile. The approach also requires constraints (written in Object Constraint Language (OCL) [27]) specified on the UAV domain model and state invariants on the flight behavior model. OCL constraints

represent the expected behavior of the SUT. A violation of an OCL constraint is considered as a deviation from the expected behavior, i.e., a violation of system specification.

Our approach automates this process by introducing an AITester in place of a human tester to perform the system-level testing of the UAS. For this purpose, we use an AI technique called deep reinforcement learning (DRL). In DRL, an AI agent tries to maximize the reward by performing different actions in the environment [28]. The environment for the DRL is commonly defined in the form of the Markov Decision Process (MDP). To formulate the problem into an MDP, our approach utilizes the UAV structural and behavioral profiles. The UAV structural profile represents an observation space of the environment. Similarly, the UAV behavioral profile represents the state and action space of the DRL algorithm. The UAV domain model serves two purposes, (i) initializing the observation space of the environment, and (ii) generating an instance model that will be used to evaluate the OCL constraints during execution. The flight behavior model of the UAV is used to initialize the DRL algorithm for the UAV under test.

As a part of our DRL algorithm, we designed Long Short-Term Memory (LSTM) network architecture because the UAV flight data holds a temporal relationship due to the highly dynamic environment. The LSTM network architecture used in our approach consists of an input layer, three LSTM layers with a hidden dimension of 10, and an output layer. The first input is the abstract flight state that corresponds to one of the states in the flight behavior model. The remaining inputs represent the UAV flight information displayed on the cockpit of the GCS, e.g., *altitude* and *airspeed*. This information is necessary for the remote pilot to decide a sequence of actions to fly the UAV successfully [29]. The output of the LSTM network is the possible actions, and the action with the highest probability is selected using the *softmax* function.

We use Deep Q-Network (DQN) algorithm [30] as our

¹<https://github.com/hassansartaj/uast-toolset>

DRL algorithm along with the LSTM network. DQN has been widely used to train different game-playing bots and to get human-level control [31], [32]. DQN is a natural choice in our case because it uses Experience Replay to avoid high correlations among a sequence of actions.

Using the UAV flight behavior model and domain model, the AITester initializes the states, actions, and observation space of the environment. Starting from the initial flight state, the AITester selects an action (e.g., *takeoff* or *land*) randomly or using the LSTM network according to the Epsilon greedy strategy. The selected action is executed on the environment using DroneKit API [33]. The function of DroneKit is to receive the action command for the UAV, send the command to the UAV using a communication link (using Mavlink), and receive the state of the UAV. The continuous change in the flight state of the UAV is monitored by a state observer. This helps the AITester to keep track of the current state, action, and the next state of the UAV. The flight data observed during execution is used as a part of LSTM network input and to populate an instance model of the UAV domain model. OCL constraints and state invariants are evaluated on the instance model [34]. The reward is calculated based on the number of failed OCL constraints and the selected action. In this way, the AITester continues to explore different flight states, learns to violate OCL constraints based on actions, and compiles the results for further analysis by an avionics tester. At the end of execution, results compiled in the form of a test report are obtained. The generated test report contains information related to UAV flight states, actions, and the number of failed OCL constraints for each execution.

VI. PILOT EXPERIMENT

For the preliminary evaluation of our proposed approach, we perform a pilot experiment using ArduCopter as a case study. ArduCopter is a widely used open-source Ardupilot [35] project. It supports a variety of multi-copter UAVs and autopilot hardware. It has also been used in many experiments [36].

A. Research Questions

We formulate two research questions for this evaluation.

RQ1: *Is the proposed approach effective in violating expected behavior of SUT?*

Motivation: The main goal of our approach (AITester) is to identify such flight scenarios (by executing correct actions in different flight states) that can lead to violating the maximum number of expected conditions of SUT specified in the form of OCL constraints. Thus, RQ1 aims to analyze the effectiveness of our approach in violating OCL constraints while performing an accurate sequence of actions.

RQ2: *Is the proposed approach efficient in learning the UAV flight behavior?*

Motivation: Since our approach minimizes the manual effort in testing UAS, the purpose of RQ2 is to evaluate the efficiency in terms of execution time required for accurately learning the UAV flight behavior.

B. Experiment Setup and Execution

We use the proposed UAV profiles to model the domain and behavior of the ArduCopter with the UAV type of quadcopter. The domain model and OCL constraints are modeled by analyzing the source code of ArduCopter. We use the Ardupilot software-in-the-loop simulator that is built using the binaries of the same version of the ArduCopter source code. The complete experiment setup is available along with the toolset¹.

To compare AITester for the analysis of research questions (RQ1 and RQ2), we use Random testing as a baseline which is common practice for such experiments, e.g., in [37]. We execute AITester and Random for 1000 epochs. Both AITester and Random are executed on two machines with the exact same specifications, i.e., core i7 3.6 GHz processor, 64 GB RAM, 1 TB hard drive, NVIDIA GeForce® RTX 2080 Ti graphics card, and Windows 10 64-bit operating system. The hyperparameters are selected based on the different experimental trials because there are no standard guidelines available for this purpose.

C. Experiment Results

The results show that AITester is able to violate a maximum number (≈ 40 out of 48) of OCL constraints while executing actions accurately (accuracy $> 90\%$). In the case of Random, the number of failed OCL constraints are less than half (≈ 20) of the total number (48) of constraints with an average accuracy of $\approx 60\%$.

RQ1 Result: AITester is effective in violating the expected behavior of SUT with more than 90% accuracy.

The results show that the execution time of the AITester varies from one minute to ≈ 20 minutes with an average accuracy of $\approx 91\%$. Whereas, in the case of Random, time increases from one minute to ≈ 40 minutes with an average accuracy of $\approx 60\%$.

RQ2 Result: AITester is time-efficient as it takes ≈ 14 minutes for the end-to-end testing activity.

VII. CONCLUSION AND FUTURE PLAN

In this research work, we propose an approach to automate the system-level testing of the UAS. The proposed approach utilizes MBT and AI techniques to create an AITester that generates, executes, and evaluates test scenarios based on the environmental context at runtime. We provide a toolset to support testing automation. We perform a pilot study to analyze the preliminary results of the proposed approach.

In the future, we plan to conduct a detailed empirical study to evaluate our proposed approach from two perspectives. First, we plan to evaluate different UAV autopilots for SIL/HIL testing. Second, we plan to extend our approach for testing cockpit display systems (CDS) of GCS [29] and integrate the toolset with our previous CDST toolkit [38]. Lastly, we plan to evaluate our approach using the CDS of GCS as SUT.

ACKNOWLEDGMENT

This research work is supervised by Dr. Muhammad Zohaib Iqbal and supported through a research grant titled ‘Establishment of National Center of Robotics and Automation (NCRA)’ by Higher Education Commission (HEC) Pakistan.

REFERENCES

- [1] T. K. Ferrell and U. D. Ferrell, “Rtca do-178c/eurocae ed-12c,” *Digital Avionics Handbook*, 2017.
- [2] M. M. Marques, “Stanag 4586—standard interfaces of uav control system (ucs) for nato uav interoperability,” *NATO Standardization Agency: Afeite, Portugal*, p. 14, 2012.
- [3] D. Jung and P. Tsiotras, “Modeling and hardware-in-the-loop simulation for a small unmanned aerial vehicle,” in *AIAA Infotech@ Aerospace 2007 Conference and Exhibit*, 2007, p. 2768.
- [4] J. P. How, B. Behnhke, A. Frank, D. Dale, and J. Vian, “Real-time indoor autonomous vehicle test environment,” *IEEE Control Systems Magazine*, vol. 28, no. 2, pp. 51–64, 2008.
- [5] Y. Wu, L. Qi, Z. Cheng, Y. Xiao, Z. Yang, W. Liu, and X. Li, “Testing method and anti-drift capability analysis of downwash airflow distribution characteristics of multi-rotor uav in hover,” in *2019 ASABE Annual International Meeting*. American Society of Agricultural and Biological Engineers, 2019, p. 1.
- [6] M. Scanavino, A. Vilardi, and G. Guglieri, “A new facility for uav testing in climate-controlled environments,” in *2019 International Conference on Unmanned Aircraft Systems (ICUAS)*. IEEE, 2019, pp. 1436–1444.
- [7] E. N. Johnson and S. Fontaine, “Use of flight simulation to complement flight testing of low-cost uavs,” in *AIAA Modeling and Simulation Technologies Conference*, 2001.
- [8] E. Sorton and S. Hammaker, “Simulated flight testing of an autonomous unmanned aerial vehicle using flightgear,” in *Infotech@ Aerospace*, 2005, p. 7083.
- [9] K. Sevcik and P. Oh, “Testing unmanned aerial vehicle missions in a scaled environment,” in *Unmanned Aircraft Systems*. Springer, 2008, pp. 297–305.
- [10] G. E. Mullins, P. G. Stankiewicz, and S. K. Gupta, “Automated generation of diverse and challenging scenarios for test and evaluation of autonomous vehicles,” in *2017 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2017, pp. 1443–1450.
- [11] P. Schulte and D. Spencer, “On-board model-based fault diagnosis for autonomous proximity operations,” in *69th International Astronautical Congress, Bremen, Germany*, 2018.
- [12] T. Ma, S. Ali, and T. Yue, “Modeling foundations for executable model-based testing of self-healing cyber-physical systems,” *Software & Systems Modeling*, pp. 1–31, 2018.
- [13] P. Ulbig, D. Müller, C. Torens, C. C. Insaurralde, T. Stripf, and U. Durak, “Flight simulator-based verification for model-based avionics applications on multi-core targets,” in *AIAA Scitech 2019 Forum*, 2019, p. 1976.
- [14] M. Sarkar, A. Homaifar, B. A. Erol, M. Behniapoor, and E. Tunstel, “Pie: a tool for data-driven autonomous uav flight testing,” *Journal of Intelligent & Robotic Systems*, pp. 1–18, 2019.
- [15] C. De Wagter, B. Remes, R. Ruisink, F. Van Tienen, and E. Van der Horst, “Design and testing of a vertical take-off and landing uav optimized for carrying a hydrogen fuel-cell with pressure tank,” in *International Micro Air Vehicle Conference and Competition (IMAV)(Madrid)*, 2019.
- [16] M. Rizk, A. Mroue, K. A. Ali, and J. Charara, “Development and implementation of an embedded low-cost gnss receiver emulator for uav testing,” in *2019 26th IEEE International Conference on Electronics, Circuits and Systems (ICECS)*. IEEE, 2019, pp. 101–102.
- [17] M. Yang, D. Darrah, J. Eppler, W. Liu, and W. A. Anemaat, “Small uav acoustic design, analysis and testing,” in *AIAA Scitech 2019 Forum*, 2019, p. 1073.
- [18] L. Ambroziak and Z. Gosiewski, “Preliminary uav autopilot integration and in-flight testing,” in *Solid State Phenomena*, vol. 198. Trans Tech Publ, 2013, pp. 232–237.
- [19] D. Erdos and S. E. Watkins, “Uav autopilot integration and testing,” in *2008 IEEE Region 5 Conference*. IEEE, 2008, pp. 1–6.
- [20] R. F. Hartley, F.-D. X. Hugon, R. Anderson, and H. Moncayo, “Development and flight testing of a model based autopilot library for a low cost unmanned aerial system,” in *AIAA Guidance, Navigation, and Control (GNC) Conference*, 2013, p. 5009.
- [21] B. Gafford, T. Dürschmid, G. A. Moreno, and E. Kang, “Synthesis-based resolution of feature interactions in cyber-physical systems,” in *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*, 2020, pp. 1090–1102.
- [22] J. Junell, T. Mannucci, Y. Zhou, and E.-J. Van Kampen, “Self-tuning gains of a quadrotor using a simple model for policy gradient reinforcement learning,” in *AIAA Guidance, Navigation, and Control Conference*, 2016, p. 1387.
- [23] M. Siddiquee, J. Junell, and E.-J. Van Kampen, “Flight test of quadcopter guidance with vision-based reinforcement learning,” in *AIAA Scitech 2019 Forum*, 2019, p. 0142.
- [24] T. Ma, S. Ali, T. Yue, and M. Elaasar, “Testing self-healing cyber-physical systems under uncertainty: a fragility-oriented approach,” *Software Quality Journal*, pp. 1–35, 2019.
- [25] M. Utting and B. Legeard, *Practical model-based testing: a tools approach*. Elsevier, 2010.
- [26] OMG, “Uml unified modeling language specification, version 2.5.1,” *Object Management Group Inc.*, 2017. [Online]. Available: <http://www.omg.org/spec/UML/2.5.1/>
- [27] —, “Object constraint language specification v2.4,” *Object Management Group Inc.*, 2014. [Online]. Available: <http://www.omg.org/spec/OCL/2.4/>
- [28] S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*. Prentice Hall, 2009.
- [29] H. Sartaj, M. Z. Iqbal, and M. U. Khan, “Testing cockpit display systems of aircraft using a model-based approach,” *Software and Systems Modeling*, pp. 1–26, 2021.
- [30] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski et al., “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [31] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller, “Playing atari with deep reinforcement learning,” *arXiv preprint arXiv:1312.5602*, 2013.
- [32] M. Jaderberg, W. M. Czarnecki, I. Dunning, L. Marris, G. Lever, A. G. Castaneda, C. Beattie, N. C. Rabinowitz, A. S. Morcos, A. Ruderman et al., “Human-level performance in 3d multiplayer games with population-based reinforcement learning,” *Science*, vol. 364, no. 6443, pp. 859–865, 2019.
- [33] DroneKit, “Dronekit python,” *Open source community*, 2015. [Online]. Available: <https://github.com/dronekit/dronekit-python>
- [34] H. Sartaj, M. Z. Iqbal, A. A. A. Jilani, and M. U. Khan, “A search-based approach to generate mc/dc test data for ocl constraints,” in *International Symposium on Search Based Software Engineering*. Springer, 2019, pp. 105–120.
- [35] ArduPilot, “Ardupilot project,” *ArduPilot community*, 2020. [Online]. Available: <https://ardupilot.org/ardupilot/index.html>
- [36] H. Lee and H. J. Kim, “Trajectory tracking control of multirotors from modelling to experiments: A survey,” *International Journal of Control, Automation and Systems*, vol. 15, no. 1, pp. 281–292, 2017.
- [37] Y. Zheng, X. Xie, T. Su, L. Ma, J. Hao, Z. Meng, Y. Liu, R. Shen, Y. Chen, and C. Fan, “Wuji: Automatic online combat game testing using evolutionary deep reinforcement learning,” in *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2019, pp. 772–784.
- [38] H. Sartaj, M. Z. Iqbal, and M. U. Khan, “Cdst: A toolkit for testing cockpit display systems,” in *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)*. IEEE, 2020, pp. 436–441.